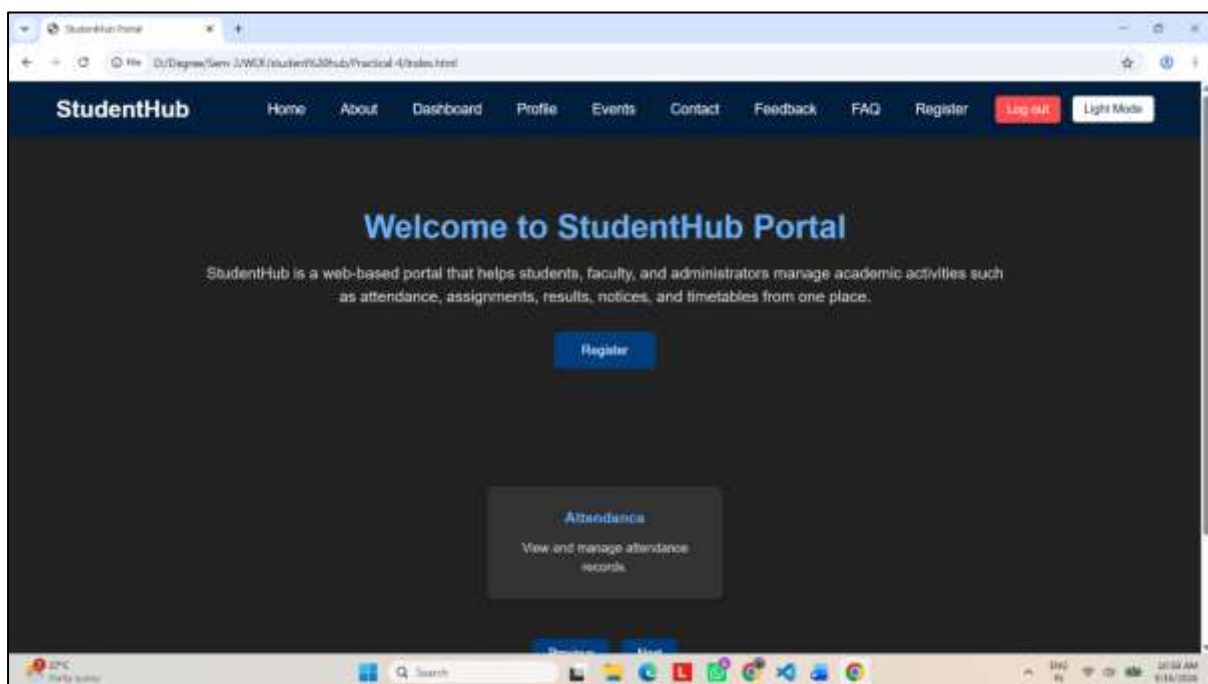
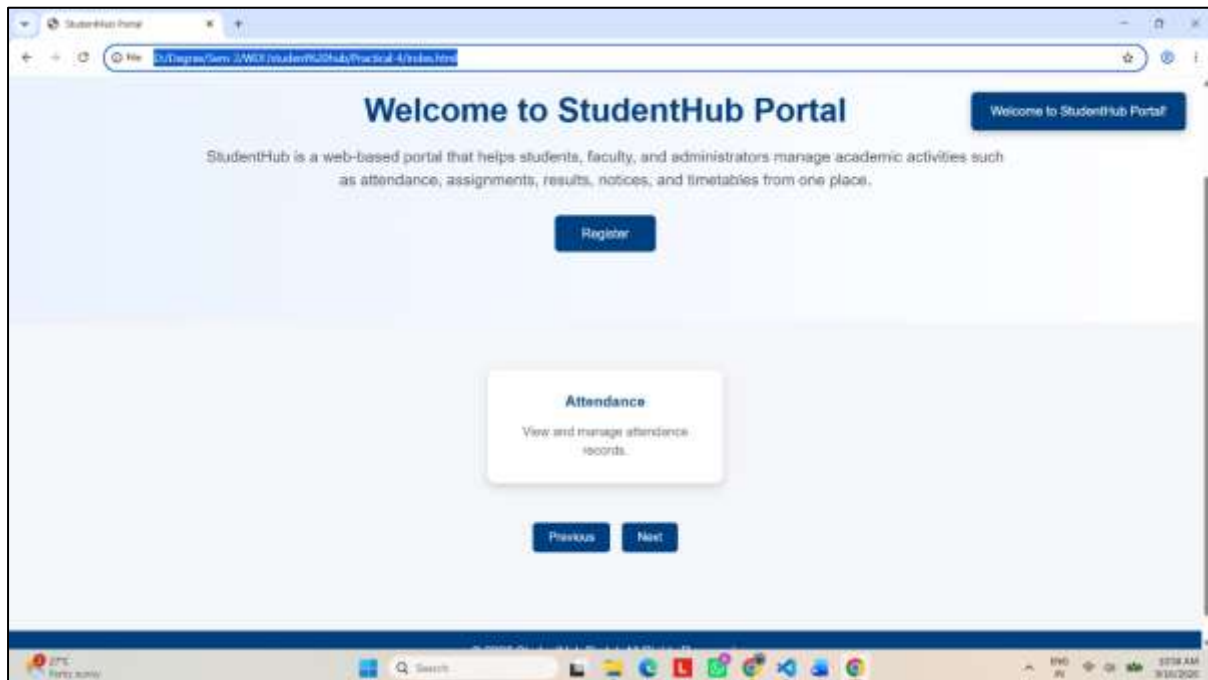
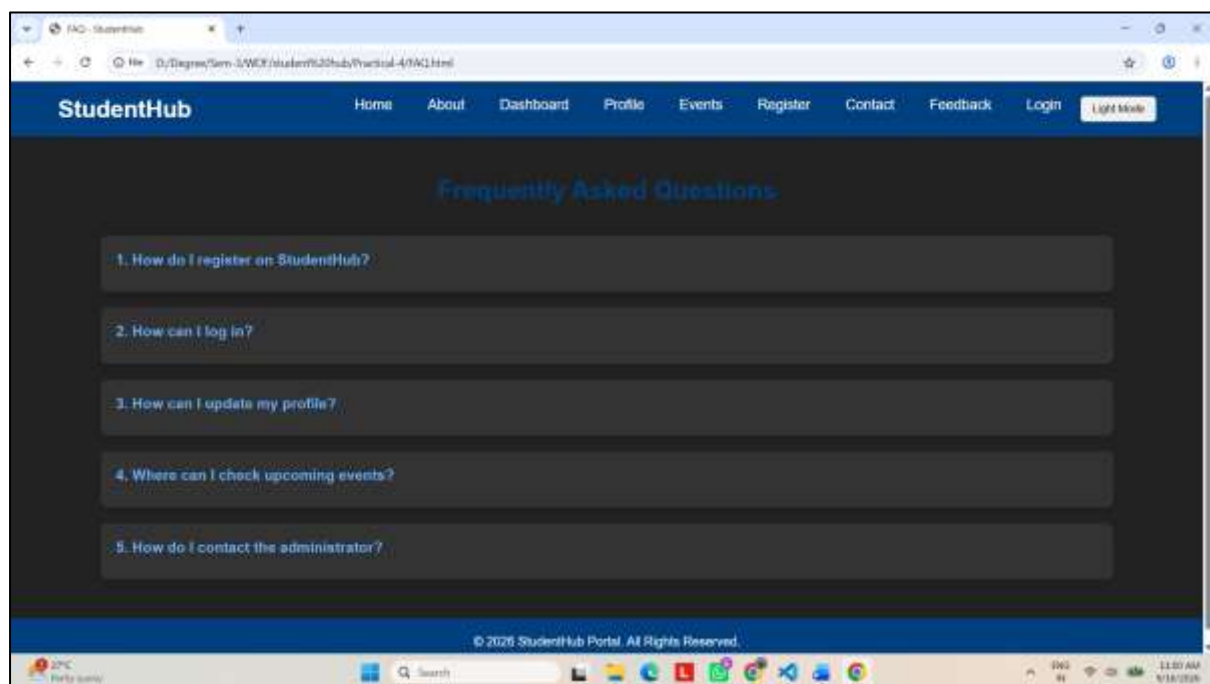
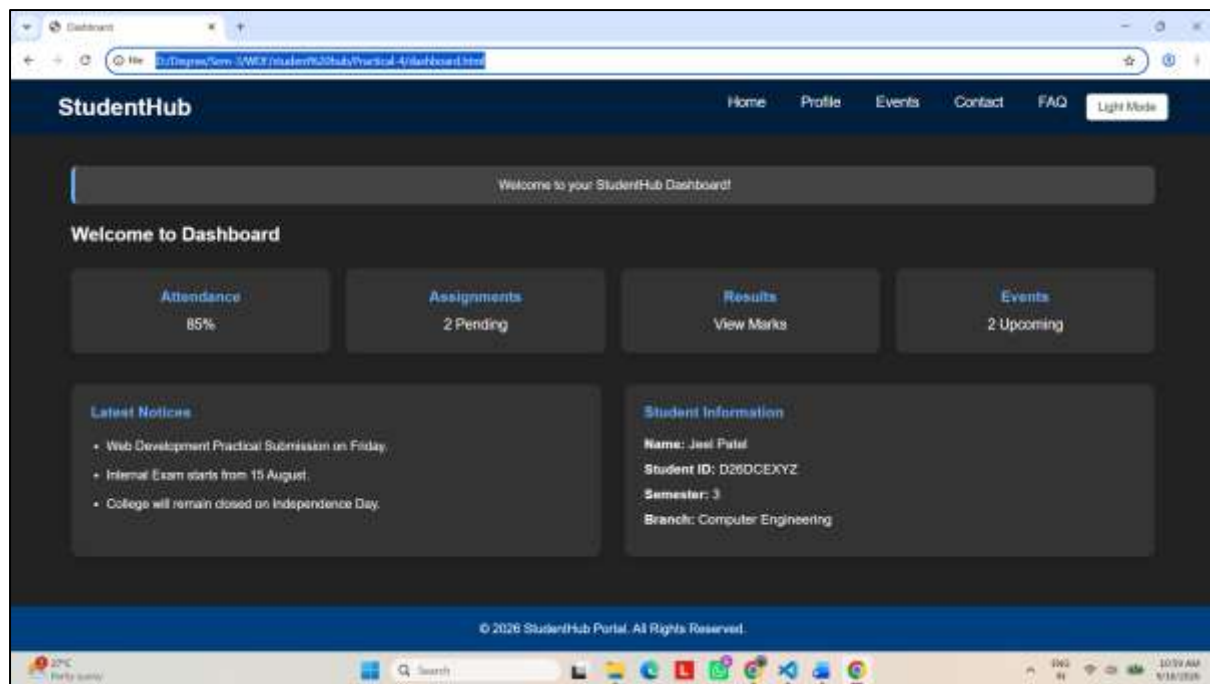


Practical – 4

Practical Aim:

Add dynamic UI components such as collapsible FAQ, modal popup, image/content slider, notification banner, hamburger menu, and light/dark theme switcher using JavaScript.





Key Questions

1. How are DOM elements selected and modified?

DOM elements are selected using methods like `getElementById()` and `querySelector()`. They are modified using properties such as `innerHTML`, `style`, and `classList`.

2. Are event listeners attached correctly?

Yes. Event listeners are attached using `addEventListener()`. They handle events such as button clicks and FAQ question clicks correctly.

3. Is `localStorage` used for remembering theme choice?

Yes. `localStorage.setItem()` saves the selected theme, and `localStorage.getItem()` retrieves it when the page is opened again.

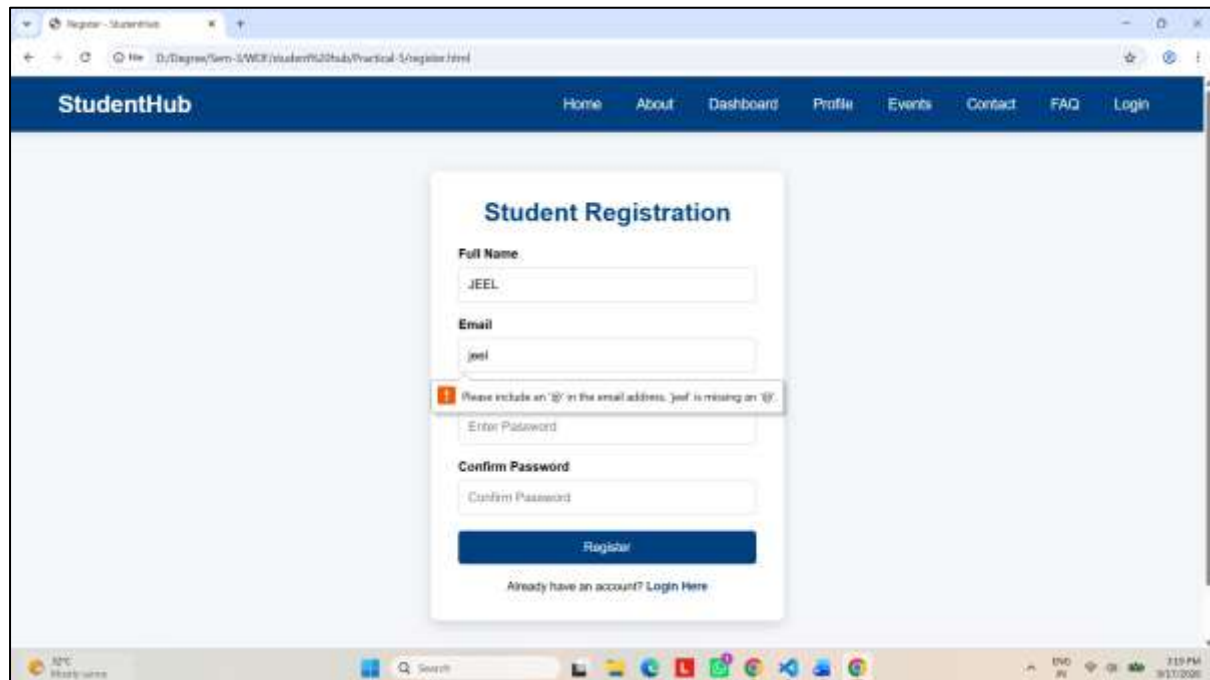
4. Does interactivity improve usability without breaking accessibility?

Yes. Interactive features such as collapsible FAQs, theme switching, notifications, and popups make the website easier to use while keeping the content accessible and simple to operate.

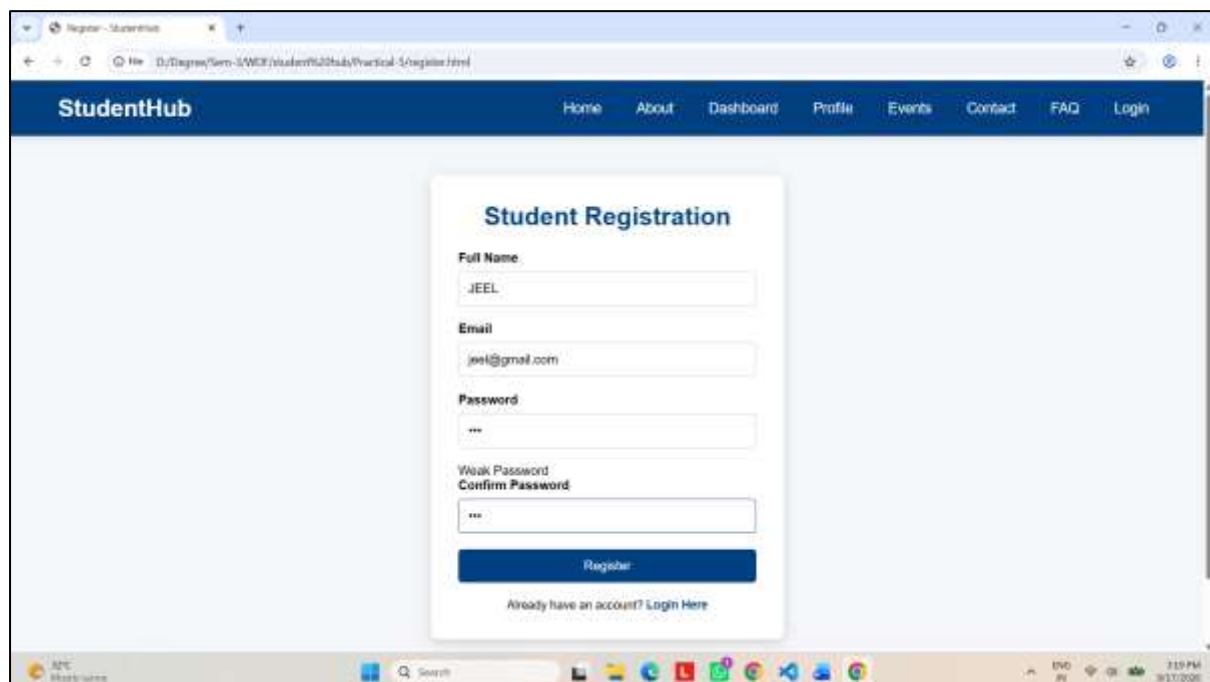
Practical – 5

Practical Aim:

Create a student registration form with HTML5 input types and JavaScript validation for name, email, mobile number, password, confirm password, course, year, gender, and terms acceptance. Use Regular Expression for validations.



The screenshot shows a web browser window with the URL `D:\Degree\Sem-5\WDF\studentHub\Practical-5\register.html`. The page has a dark blue header with the text "StudentHub" and navigation links: Home, About, Dashboard, Profile, Events, Contact, FAQ, and Login. The main content area features a "Student Registration" form. The form fields are: Full Name (containing "JEEL"), Email (containing "jeel"), Enter Password, and Confirm Password. A red error message is displayed below the email field: "Please include an '@' in the email address, 'jeel' is missing an '@'". A blue "Register" button is at the bottom of the form, and a link "Already have an account? Login Here" is below it. The Windows taskbar at the bottom shows the date and time as 3:17:00 PM.



The screenshot shows the same "Student Registration" form, but the email field now contains "jeel@gmail.com". The error message is no longer present. The "Enter Password" field is now labeled "Password" and contains three asterisks. The "Confirm Password" field is also labeled "Confirm Password" and contains three asterisks. The "Register" button and the "Login Here" link remain at the bottom of the form. The Windows taskbar at the bottom shows the date and time as 3:17:00 PM.

The screenshot shows a web browser window with the address bar displaying 'D:\Degree\Sem-5\WCE\studentHub\Practical-3\register.html'. The page title is 'StudentHub'. The main content area features a 'Student Registration' form. The form has the following fields and elements:

- Full Name:** A text input field containing the value 'JEEL'.
- Email:** A text input field containing the value 'jeel@gmail.com'.
- Password:** A password input field with a validation error message 'This page says: Enter a strong password' displayed above it.
- Confirm Password:** A text input field.
- Register:** A blue button at the bottom of the form.
- Login Here:** A link below the Register button.

The browser's taskbar at the bottom shows the system clock as 2:19 PM on 3/17/2020.

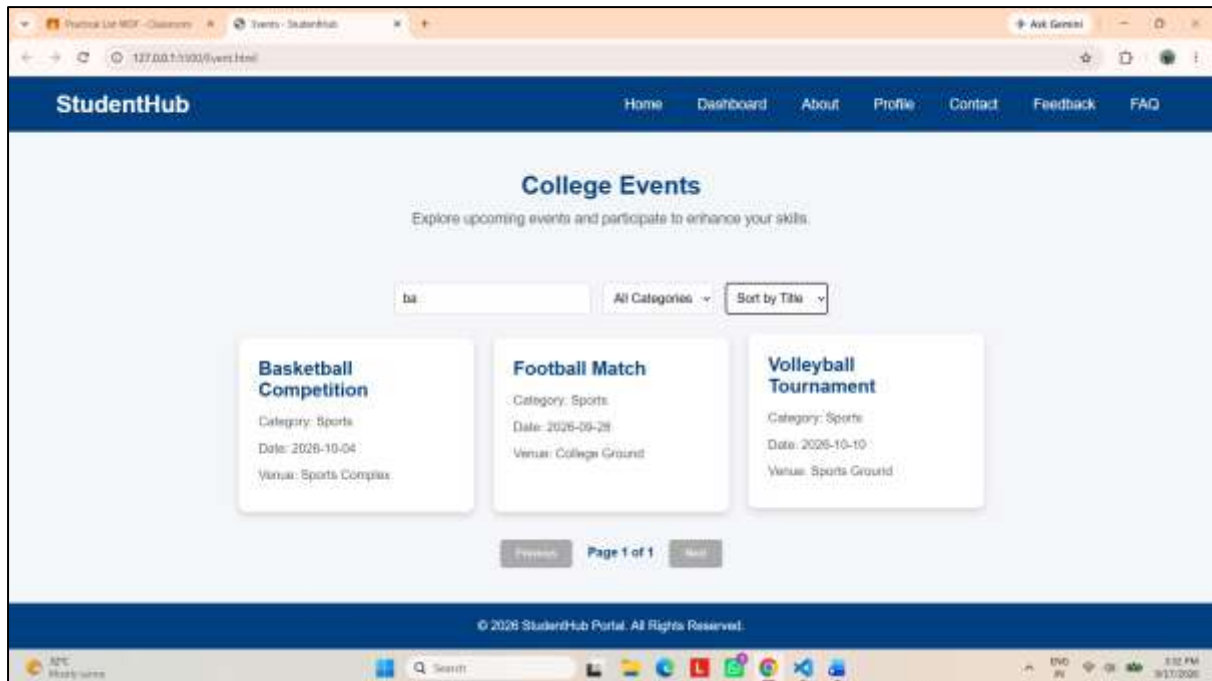
Key Questions:-

- 1. Are correct input types and attributes used?**
Yes. Appropriate input types such as email, password, text, and required attributes are used.
- 2. Are validation errors displayed near the relevant fields?**
Yes. Validation error messages are displayed near the corresponding input fields so users can easily identify and correct them.
- 3. Is password strength checked?**
Yes. The password is checked for basic requirements such as minimum length and a combination of characters.
- 4. Is the form accessible using keyboard and screen-reader-friendly labels?**
Yes. The form can be navigated using the keyboard, and proper <label> elements are provided for screen-reader accessibility.

Practical – 6

Practical Aim:

Display event lists, student profiles, notices, or FAQs from external JSON files using Fetch API. Implement dynamic rendering, search, filter, sorting, and pagination.



Key Questions

1. How is JSON fetched, parsed, and rendered?

JSON data is fetched using the **fetch()** method. The response is converted into JSON using **response.json()**. After parsing, the data is stored in an array and displayed on the webpage by creating HTML elements using JavaScript.

Flow:

fetch() → response.json() → store data → create HTML → display on page

2. Which array methods are used for search, filter, sort, and pagination?

- Search: includes() or find()
- Filter: filter()
- Sort: sort()
- Pagination: slice()

3. How are loading and error states handled?

A loading message is displayed while the JSON data is being fetched. If the request fails, the `catch()` block displays an error message to the user.

4. How is modularity maintained in JavaScript files?

Modularity is maintained by dividing JavaScript code into separate files or functions, where each file/function performs a specific task. For example, one file can handle API fetching, another can handle filtering, and another can handle UI rendering.